

Setup de entorno uv de cero para correr notebooks de clase

Vamos a adecuarnos a la estructura mono repo y mantener todas las dependencias del proyecto de manera homogénea.

En primer lugar, vamos a eliminar aquellos entornos locales/por proyecto para pasar a trabajar con un entorno de Python en la raíz si los hubiera. Por ejemplo:

```
rm -rf cml_project/.venv mi_entorno
# Borremos también el entorno global si lo hubieramos construido
# anteriormente, así lo haremos todo de cero
rm -rf .venv
```

A continuación vamos a asegurarnos que nuestro pyproject.toml de la raíz tenga registrados los distintos packages, debemos ver algo similar a esto:

```
[tool.uv.workspace]
members = [
    "airflow_project",
    "databricks_project",
    "cml_project",
    "notebooks_clase",
]
```

Lo que nos indica que cada paquete está referenciado. Una vez hecho esto, reconstruyamos el entorno a partir del uv.lock:

```
uv sync --locked --all-packages
```

Podríamos ejecutar seleccionando el package particular:

```
uv run --package cml_project python cml_project/src/train.py
uv run --package databricks_project python
databricks_project/ML_Lab_MLflow_Databricks.py
```

Error típico

Trato de agregar una librería (`mlflow`) pero las dependencias globales no me lo permiten al no disponer del mismo versionado:

```

uv add --package notebooks_clase mlflow
x No solution found when resolving dependencies for split
(markers: python_full_version == '3.11.*' and sys_platform ==
'darwin'):
→ Because cml-project depends on pandas>=1.5,<2 and
notebooks-clase depends on pandas>=2.2.3,<3.0.0, we
can conclude that cml-project and notebooks-clase are
incompatible.
And because your workspace requires cml-project and
notebooks-clase, we can conclude that your workspace's
requirements are unsatisfiable.

hint: While the active Python version is 3.10, the resolution
failed for other Python versions supported by your project.
Consider limiting your project's supported Python versions
using 'requires-python'.
help: If you want to add the package regardless of the failed
resolution, provide the '--frozen' flag to skip locking
and syncing.

```

Error al incluir una nueva dependencia

Solución

Resuelve los conflictos uno a uno. En la mayoría de casos, hay librerías que han quedado rezagadas y simplemente basta con actualizar. Si existe la opción de que algo no se pueda actualizar por una verdadera incompatibilidad, puedes sacarlo del monorepo, es decir, que no sea un miembro del workspace global y tratarlo por separado. En nuestro caso, hay una librería denominada `tts` en telegram project que depende de una versión de pandas antigua. Lo trataremos como proyecto por separado conociendo la limitación.

Una vez resueltas las dependencias

Puedes activar el entorno:

```

source .venv/bin/activate
# 0 su equivalente en Windows

```

Y revisar si se han instalado correctamente:

```

uv pip list --python .venv/bin/python
# Otra opción
.venv/bin/python -c "import mlflow; print(mlflow.__version__)"

```

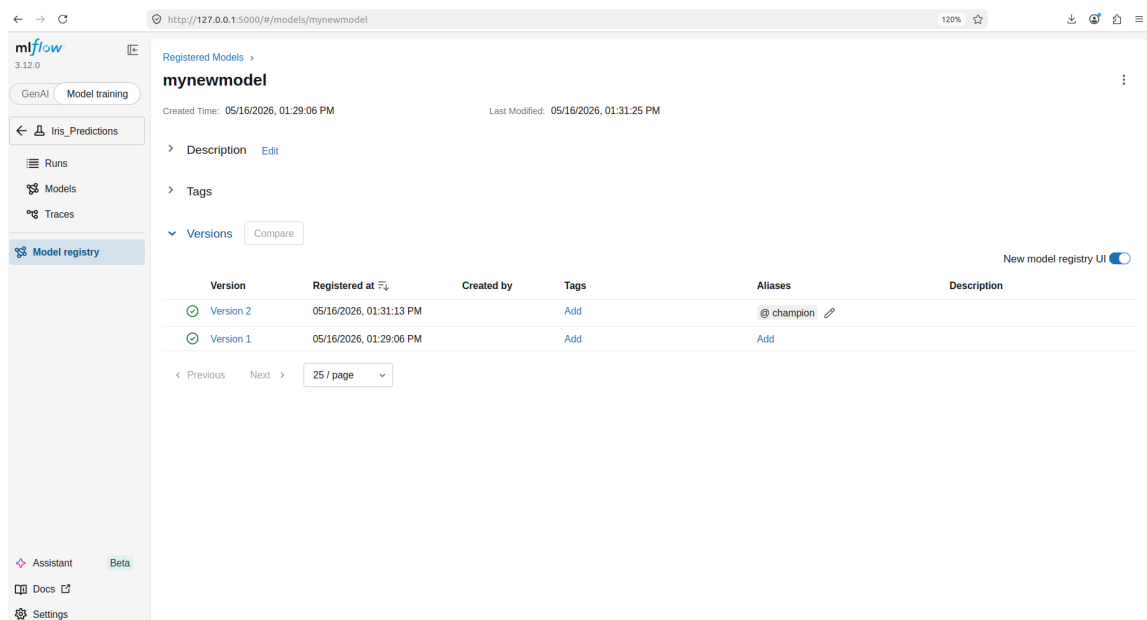
Evita utilizar `pip` directamente sobre el entorno.

Nota: subidas todas las versiones de python a 3.11 desde `requires-python` en los `pyproject.toml`

Ejecución de la práctica de mlflow

Una vez hayáis construido el entorno podréis continuar con la práctica siguiendo los pasos:

- En (./chapter-3-mlflow.ipynb) lanzaremos los primeros experimentos (con y sin autolog). En clase no apuntamos correctamente a `mlruns`, es muy importante apuntar a la carpeta correcta donde guardar los experimentos, tienes más detalle en el Notebook. Para chequear que la ejecución es correcta, dentro de cada experimento deberá existir el archivo `meta.yaml`.
- En (./chapter-3-1-mlflow-tracking.ipynb) correremos una instancia de mlflow ui, importante el punto anterior, tenéis los comandos para ejecutar tanto en bash como en powershell.
- En (./chapter-3-2-mlflow-experiments.ipynb) revisaréis como lanzar experimentos con cross validation y comparar resultados. Además, una ejecución end-to-end con un modelo xgboost con sus particularidades.
- Tanto en (./chapter-3-3-mlflow-models.ipynb) como (./chapter-3-4-model-registry.ipynb) tenéis más detalle sobre lo que implica un registro de modelo, hacedlo, registrad uno de los experimentos (recomiendo el de xgboost) y luego pasad al notebook (./chapter-3-5-model-serving).
- Por último, se espera que en (./chapter-3-5-model-serving) seáis capaces de listar los modelos registrados y servir y consumir uno de ellos.



Ejemplo de modelo registrado con dos versiones, la segunda el champion